

# Spherepop OS:

## A Deterministic Semantic Operating System

Formal Specification, Architectural Rationale, and Foundations of Pre-Linguistic Intelligence

Flyxion

December 14, 2025

### Abstract

Spherepop OS is a deterministic semantic operating system whose primary abstraction is neither the process, nor the file, nor the thread, but an append-only, causally ordered event substrate from which all semantic state is derived. The system is designed to support collaborative, time-aware, and introspectable computation by enforcing strict replayability, total causal order, and a principled separation between authoritative state transitions and non-authoritative views.

This paper presents a comprehensive formal specification of Spherepop OS together with the architectural rationale underlying each design decision. Beyond operating-system concerns, the paper situates Spherepop OS within a broader theoretical context, arguing that its event-sourced semantics instantiate foundational principles of intelligence articulated by Yi Ma and others: parsimony, self-consistency, and abstraction grounded in structure rather than language.

By distinguishing compression as reduction in causal construction from abstraction as reduction in semantic representatives, Spherepop OS resolves a longstanding ambiguity in the theory of intelligence. The resulting architecture functions as a pre-linguistic semantic substrate, positioned at the level of animal cognition rather than symbolic reasoning. In this sense, Spherepop OS occupies an intermediate position between an operating system kernel, a distributed database, and a field-theoretic model of intelligence grounded in time, construction, and equivalence.

## 1 Motivation and Design Orientation

Modern operating systems are historically shaped by assumptions that no longer hold in collaborative, distributed, or semantically rich computational environments. The dominant abstractions of processes, address spaces, and mutable global state emerged in an era where computation was primarily local, sequential, and opaque. While these abstractions have proven effective for managing hardware resources, they impose severe limitations on reproducibility, introspection, and principled reasoning about time.

In particular, traditional systems make it difficult to answer questions such as: *What exactly happened to produce the current state? Can that state be reconstructed independently? Can multiple observers reason about the same history without interfering with one another?* These difficulties

are not accidental but structural, arising from the use of mutable state, implicit causality, and side-effect-laden control flow.

Spherepop OS is motivated by the conviction that these limitations can only be resolved by elevating time and construction to first-class concepts. Rather than treating history as an implementation detail, Spherepop OS treats it as the sole source of authority. All semantic state exists only as the deterministic consequence of replaying an immutable, totally ordered event log.

This orientation aligns with a deeper theoretical perspective on intelligence. As emphasized by Yi Ma, intelligent systems are governed by two foundational principles: parsimony and self-consistency. Parsimony demands that the system explain the world using as little structure as necessary, while self-consistency demands that the resulting model be rich enough to reliably recreate and simulate its environment. Spherepop OS embeds these principles architecturally rather than heuristically: every unnecessary construction leaves a permanent trace, and every inconsistency is exposed through replay.

The design goals of Spherepop OS therefore follow not from performance considerations but from epistemic ones. The system is designed to guarantee deterministic replay, enforce a single global causal order, separate causes from observations, privilege semantic relations over procedural artifacts, and support collaborative introspection without destabilization. These commitments collectively rule out large classes of conventional kernel, database, and version-control architectures, motivating the log-centric design developed in the sections that follow.

## 2 Core Ontology

The ontology of Spherepop OS is intentionally minimal. Rather than introducing a rich taxonomy of object types, the system defines a small set of primitives from which all higher-level structure is derived.

**Definition 2.1** (Semantic Object). *A semantic object is an abstract entity identified by a stable handle. Semantic objects possess no intrinsic meaning, internal state, or behavior. Their significance arises solely through participation in relations, equivalence classes, and events.*

This definition reflects a deliberate refusal to ascribe semantics at the object level. Objects are not records, files, or processes; they are structural placeholders whose interpretation emerges only through context.

**Definition 2.2** (Relation). *A relation is a typed, directed association between two semantic objects. Relations are first-class entities and may carry auxiliary metadata, but their semantic force derives from their position within the relational structure induced by the event log.*

**Definition 2.3** (Equivalence). *An equivalence relation over semantic objects identifies multiple handles as representing a single canonical representative. Equivalence is induced explicitly by merge events and maintained via a union-find structure during replay.*

Crucially, none of these entities exist independently of the event log. There is no mutable object table, no hidden registry of relations, and no authoritative state outside replay. This ensures that the ontology of the system is entirely historical: to exist is to have been constructed.

### 3 The Authoritative Event Log

**Axiom 3.1** (Append-Only Authority). *All authoritative state transitions in Spherepop OS are recorded as events in a single append-only log. No other structure is permitted to introduce, modify, or erase semantic state.*

Each event is assigned a strictly increasing event identifier by a single arbiter, establishing a total causal order over all state transitions. This total order is not an emergent property but an enforced invariant.

**Proposition 3.1** (Deterministic State Reconstruction). *Given an initial empty state and any finite prefix of the event log, the resulting semantic state is uniquely determined.*

*Proof.* All kernel transitions are pure functions of prior state and the next event in the log. Since the log is immutable and totally ordered, replay yields a unique result.  $\square$

This proposition elevates the event log from a storage mechanism to the primary semantic artifact of the system. The log is not merely a record of what happened; it *is* what happened.

In the context of intelligence theory, this construction has a significant implication. Compression in Spherepop OS is not an abstract information measure but a concrete reduction in causal construction. Abstraction, as will be shown later, is not symbolic generalization but structural equivalence over constructed entities. Both are directly visible in the log, rendering the system introspectable in a way that purely statistical models are not.

### 4 Kernel Semantics and Deterministic Execution

At the heart of Spherepop OS lies a deliberately minimal kernel. Unlike traditional kernels, which manage processes, memory, and scheduling, the Spherepop kernel performs a single task: it deterministically interprets the authoritative event log. All higher-level phenomena—state, structure, abstraction, and even time—emerge from this interpretation.

**Definition 4.1** (Kernel). *The kernel is the deterministic transition function that maps an event-log prefix to a semantic state. It possesses no mutable internal state beyond that which is derivable from replay, and it produces no side effects beyond derived views.*

This definition is intentionally austere. The kernel is not a scheduler, a memory manager, or a policy engine. It is a semantic interpreter whose sole responsibility is to ensure that identical histories yield identical worlds.

This design enforces self-consistency in the strongest possible sense. Because the kernel is pure, replayable, and free of hidden state, the system is always capable of reconstructing its own present from first principles. There is no distinction between *how* the state was reached and *what* the state is; the former fully determines the latter.

## 4.1 Derived State and Non-Authority

Although the kernel maintains derived structures—such as equivalence classes, relation tables, and metadata maps—these structures are explicitly non-authoritative. They exist only as caches that accelerate observation. At any time, they may be discarded and reconstructed by replay.

This distinction is essential. In conventional systems, caches frequently become sources of truth through optimization or necessity. In Spherepop OS, by contrast, authority is rigidly centralized in the log. The kernel is therefore immune to a wide class of synchronization errors and semantic drift.

## 4.2 Kernel State Formalization

To make these claims precise, we formalize the kernel state.

**Definition 4.2** (Kernel State). *A kernel state is a tuple*

$$\sigma = (O, U, R, M),$$

where  $O$  is a finite set of object identifiers,  $U : O \rightarrow O$  is a union-find parent map encoding equivalence,  $R \subseteq O \times O \times \mathcal{T}$  is a set of typed relations, and  $M : O \rightarrow \mathcal{K} \rightarrow \mathcal{V}$  is a partial metadata map.

We write  $\text{rep}_\sigma(o)$  for the canonical representative of object  $o$  under the equivalence relation induced by  $U$ . Path compression is applied deterministically during replay.

## 4.3 Operational Semantics of Events

Kernel execution is defined by a labeled transition relation

$$\sigma \xrightarrow{e} \sigma',$$

where  $e$  is an event drawn from a fixed and closed set of event types. Each event type corresponds to a semantic commitment that refines the structure of the world.

**Object Introduction (POP).** A POP event introduces a new semantic object into existence. The object has no properties beyond its identity and initial self-representation under equivalence. This reflects the principle that objects are inert until related.

Formally, if  $o \notin O$ , then

$$(O, U, R, M) \xrightarrow{\text{POP}(o)} (O \cup \{o\}, U[o \mapsto o], R, M).$$

**Equivalence Induction (MERGE).** A MERGE event identifies two existing objects as equivalent. Importantly, this operation does not destroy either object; instead, it collapses them into a shared representative.

If  $o_1, o_2 \in O$ , then

$$\sigma \xrightarrow{\text{MERGE}(o_1, o_2)} \sigma',$$

where the representative of  $o_2$  is redirected to that of  $o_1$ .

This operation is idempotent and confluent. Applying MERGE to already equivalent objects has no effect, and the final equivalence relation is independent of merge order. These properties ensure that abstraction is stable under replay and resistant to race conditions.

**Proposition 4.1** (Confluence of Equivalence). *The equivalence relation induced by a set of MERGE events depends only on the set of merges performed, not on their order.*

*Proof.* Union-find computes the transitive closure of the merge relation, which is order-independent.  $\square$

**Relation Construction (LINK and UNLINK).** LINK events introduce directed, typed relations between canonical representatives of objects. UNLINK events remove such relations. Both operations are interpreted relative to current equivalence classes, ensuring that abstraction propagates uniformly through relational structure.

**Bulk Abstraction (COLLAPSE).** The COLLAPSE event generalizes MERGE by inducing equivalence over an entire region of objects. Conceptually, COLLAPSE corresponds to a deliberate act of abstraction: many constructed entities are retrospectively identified as instances of a single concept.

This operation is especially significant from the perspective of intelligence theory. COLLAPSE does not reduce the number of events that occurred; instead, it reduces the number of distinct representatives that matter. As such, it exemplifies abstraction without compression, a distinction that will become central later in the paper.

**Metadata Attachment (SET\_META).** SET\_META attaches auxiliary information to objects without affecting semantic structure. Metadata is explicitly non-causal: it does not influence equivalence, relations, or object existence.

**Axiom 4.1** (Metadata Orthogonality). *Metadata updates commute with all kernel transitions and have no semantic effect.*

## 4.4 Determinism and Semantic Stability

The operational semantics above satisfy strong determinism guarantees.

**Theorem 4.1** (Kernel Determinism). *For any kernel state  $\sigma$  and event  $e$ , there exists at most one state  $\sigma'$  such that  $\sigma \xrightarrow{e} \sigma'$ .*

This theorem underwrites every higher-level property of Spherepop OS. It ensures that observers, regardless of vantage point or reconstruction method, converge on the same semantic world when given the same history.

From the perspective articulated by Yi Ma, this determinism is not merely a technical convenience but a prerequisite for intelligence. A system that cannot reconstruct its own world consistently cannot form reliable models of that world. By embedding determinism into its core, Spherepop OS enforces self-consistency as an architectural invariant rather than a learned behavior.

## 5 Observation, Diffs, and Non-Interference

While the authoritative event log determines all semantic state, a practical system must permit observation without compromising determinism. Spherepop OS resolves this tension by introducing a strict separation between authoritative causation and non-authoritative observation.

**Definition 5.1** (Diff). *A diff is a derived, non-authoritative description of the semantic change induced by applying a single event to a kernel state.*

Diffs are emitted by the kernel as observational artifacts. They are never themselves logged, never replayed, and never consulted during future kernel execution. Their sole purpose is to permit observers to maintain an incremental view of semantic state without replaying the entire log.

This separation is enforced by construction rather than convention. The kernel computes diffs as a pure function of state transition, but diffs have no causal power. They may be dropped, delayed, reordered, or ignored by observers without affecting the correctness of the system as a whole.

**Axiom 5.1** (Observational Non-Interference). *Diffs have no effect on authoritative state and do not influence future kernel execution.*

This axiom eliminates an entire class of feedback-induced inconsistencies that plague conventional systems. Observation is rendered epistemic rather than ontological: it reveals what has occurred without altering what can occur.

### 5.1 Incremental Reconstruction

Observers may reconstruct semantic state incrementally by applying diffs in sequence order. This reconstruction is semantically equivalent to full replay.

**Proposition 5.1** (Incremental Equivalence). *Let  $\sigma_n$  be the kernel state obtained by replaying events  $e_1, \dots, e_n$  from the empty state. Let  $\hat{\sigma}_n$  be the observer state obtained by applying diffs  $d_1, \dots, d_n$ . Then  $\sigma_n$  and  $\hat{\sigma}_n$  are semantically equivalent.*

*Proof.* Each diff  $d_i$  is deterministically derived from the kernel transition  $\sigma_{i-1} \xrightarrow{e_i} \sigma_i$ . Because kernel transitions are pure and diffs preserve all semantic information, incremental application yields a state equivalent to replay.  $\square$

This proposition ensures that no privileged reconstruction path exists. Replay-based and diff-based observers converge on the same semantic world.

## 6 Snapshots as Derived Views

To support late-joining observers and historical inspection, Spherepop OS permits the construction of snapshots.

**Definition 6.1** (Snapshot). *A snapshot is a complete serialization of kernel state derived by replaying a prefix of the event log.*

Snapshots are derived artifacts, not authoritative records. They contain no information that cannot be reconstructed from the corresponding log prefix.

**Axiom 6.1** (Snapshot Non-Authority). *Snapshots introduce no new semantic information and have no effect on future kernel execution.*

**Proposition 6.1** (Snapshot Purity). *For any snapshot  $S_k$  derived from events  $[1..k]$ , there exists a unique kernel state  $\sigma_k$  such that  $S_k$  is a faithful serialization of  $\sigma_k$ .*

Snapshots therefore serve as epistemic conveniences rather than ontological commitments. They accelerate access but do not alter meaning.

## 6.1 Replay Equivalence

The relationship between replay, diffs, and snapshots is captured by a single unifying result.

**Theorem 6.1** (Replay Equivalence). *Let  $\ell = e_1, \dots, e_n$  be an event sequence. Let  $\sigma_{\text{replay}}$  be the state obtained by replaying  $\ell$  from the empty state. Let  $\sigma_{\text{incr}}$  be the state obtained by applying diffs  $d_1, \dots, d_n$ . Then  $\sigma_{\text{replay}} = \sigma_{\text{incr}}$  up to semantic equivalence.*

*Proof.* Both constructions apply the same deterministic kernel transitions in the same order. Since diffs are derived views and introduce no additional authority, the resulting states coincide.  $\square$

This theorem guarantees global coherence across observers and reconstruction strategies.

## 7 Seekable Time and Historical Reconstruction

A distinctive feature of Spherepop OS is explicit support for temporal navigation. Rather than treating history as opaque or irreversible, the system exposes time as a first-class query dimension.

**Definition 7.1** (Seek Operation). *A seek operation  $\text{seek}(k)$  constructs an ephemeral kernel state by replaying the event log prefix  $[1..k]$ .*

Seek operations instantiate temporary kernel instances that are isolated from the live system. They permit historical inspection, debugging, and counterfactual reasoning without side effects.

**Axiom 7.1** (Temporal Isolation). *Seek operations produce no modifications to authoritative state and emit no diffs.*

**Corollary 7.1.** *Historical inspection cannot influence present or future semantic state.*

### 7.1 Temporal Monotonicity

The authoritative event sequence is strictly monotone. No operation may decrease, rewrite, or fork the authoritative timeline.

**Proposition 7.1** (Temporal Monotonicity). *If  $i < j$ , then  $\text{eid}(e_i) < \text{eid}(e_j)$ .*

This enforces a strong notion of causality: the past is immutable, and the future is always constructed relative to a fixed history.

## 8 Self-Consistency and World Simulation

The architectural commitments described above jointly enforce a strong form of self-consistency. At any moment, the system can reconstruct its entire semantic state from first principles by replaying the event log.

**Theorem 8.1** (Strong Self-Consistency). *For any time  $t$ , the semantic state of Spherepop OS is uniquely determined by the event log prefix  $[1..t]$ .*

This property directly instantiates the second foundational principle of intelligence articulated by Yi Ma: a world model must be consistent enough to support accurate simulation. In Spherepop OS, simulation is not an approximation layered atop mutable state; it is an intrinsic consequence of replay.

Because seek operations permit reconstruction of past states, and speculative branches (introduced in the next section) permit exploration of possible futures, the system possesses an internal model of its own temporal evolution. This model operates below language and symbolism. It is not a narrative or a description, but a constructive history that can be replayed, inspected, and extended.

In this sense, Spherepop OS implements self-consistency not as a learning objective but as a structural invariant. Any deviation from consistency would manifest immediately as replay failure or divergence, rendering the system epistemically unstable. By design, such instability is impossible.

## 9 Speculative Branches and Counterfactual Evaluation

An intelligent system must be capable not only of reconstructing its past but of reasoning about possible futures. Spherepop OS addresses this requirement through the introduction of speculative branches, which permit counterfactual exploration without compromising causal integrity.

**Definition 9.1** (Speculative Branch). *A speculative branch is defined by a base event identifier  $k$  together with a client-local sequence of hypothetical events that are evaluated against the authoritative history but not committed to it.*

Speculative branches are evaluated by replaying the authoritative log prefix  $[1..k]$  and then applying the speculative events in sequence. Crucially, this process uses the same deterministic kernel as authoritative execution. Speculation is therefore not a heuristic approximation but a faithful simulation of what would occur if the proposed events were to be accepted.

**Axiom 9.1** (Speculative Isolation). *Speculative branches have no effect on authoritative state unless their events are explicitly validated and sequenced by the arbiter.*

This axiom ensures that counterfactual reasoning remains epistemic rather than ontological. Speculation explores possibilities without altering reality.

**Proposition 9.1** (Counterfactual Soundness). *Speculative evaluation preserves semantic consistency with the base history.*

*Proof.* Speculative execution reuses the same deterministic transition rules as authoritative replay. Since speculative events are isolated, they cannot introduce contradictions into the authoritative timeline.  $\square$

From the standpoint of intelligence theory, speculative branching plays a role analogous to mental simulation in biological systems. The system can evaluate the consequences of potential actions without enacting them, a capacity widely regarded as central to adaptive behavior.

## 10 Compression and Abstraction as Distinct Structural Operations

A recurring theme in contemporary discussions of artificial intelligence is the ambiguity between compression and abstraction. Yi Ma has emphasized this ambiguity as a central open problem, asking what distinguishes mere data compression from genuine understanding.

Spherepop OS provides a structural resolution to this question by making both operations explicit and orthogonal within the event-derived semantic substrate.

### 10.1 Compression as Reduction in Causal Construction

In Spherepop OS, compression is defined operationally rather than statistically.

**Definition 10.1** (Compression). *Compression is the reduction in the number of events required to construct a given semantic state.*

Compression is therefore visible directly in the event log. Two histories that yield identical semantic states but differ in event count exhibit different degrees of compression. Reuse of existing objects, avoidance of redundant constructions, and deliberate minimization of causal steps all manifest as shorter logs.

This notion of compression aligns with parsimony as articulated by Yi Ma. The system is forced to confront the cost of every construction: unnecessary operations leave permanent traces in the log. There is no opportunity to retroactively erase inefficiency.

### 10.2 Abstraction as Reduction in Semantic Representatives

Abstraction, by contrast, operates on a different structural dimension.

**Definition 10.2** (Abstraction). *Abstraction is the reduction in the number of distinct semantic representatives required to produce a given semantic effect.*

Abstraction is realized in Spherepop OS through equivalence operations. MERGE and COLLAPSE events identify multiple constructed objects as representing the same underlying entity. Importantly, abstraction does not remove events from history; it reinterprets their consequences by reducing the number of distinct representatives that matter going forward.

**Proposition 10.1** (Orthogonality of Compression and Abstraction). *Compression and abstraction are independent structural operations. One may increase without increasing the other.*

*Proof.* A system may reuse a single object many times, achieving high compression with minimal abstraction. Conversely, a system may construct many distinct objects and later collapse them into a single representative, achieving high abstraction without reducing event count. The operations act on different aspects of the event-derived structure.  $\square$

This distinction resolves Yi Ma’s question without appealing to semantics or behavior. Compression concerns *how much happened*; abstraction concerns *how many distinct things matter*. Both are visible in the log, and neither subsumes the other.

## 11 Avoiding the Language Compression Trap

Yi Ma has argued that contemporary large language models operate primarily by compressing already-compressed signals. Natural language itself is the product of extensive abstraction and parsimony, encoding generations of world modeling into symbolic form. Models trained exclusively on text therefore inherit abstraction without participating in its formation.

Spherepop OS avoids this trap by operating below language. Its primitives precede symbols, syntax, and narration. Objects are not words, relations are not predicates, and equivalence is not semantic synonymy but structural identification.

**Theorem 11.1** (Pre-Linguistic Abstraction). *All abstraction in Spherepop OS arises from explicit equivalence operations over constructed entities rather than from symbolic compression.*

*Proof.* Equivalence is induced only by MERGE and COLLAPSE events applied to objects that have been constructed through prior events. No symbolic encoding or linguistic representation is required.  $\square$

This positions Spherepop OS at the level of intelligence Yi Ma associates with animals and humans prior to language: a world model built through interaction, regularity, and structural consolidation rather than textual description.

## 12 Parsimony and Self-Consistency as Architectural Invariants

Yi Ma identifies parsimony and self-consistency as the governing principles of intelligence. In Spherepop OS, these principles are not optimization objectives or emergent behaviors; they are architectural invariants.

Parsimony is enforced by the append-only log. Every construction is accounted for, and inefficiency cannot be hidden. Self-consistency is enforced by deterministic replay. Any inconsistency would manifest as divergence between replayed and observed state, an outcome that is structurally prohibited.

**Theorem 12.1** (Invariant Satisfaction). *Spherepop OS satisfies parsimony and self-consistency by construction.*

*Proof.* Parsimony follows from the irreversibility of event construction, while self-consistency follows from deterministic replay over an immutable log.  $\square$

Together, these invariants ensure that the system’s internal model of the world remains both minimal and reliable, capable of supporting simulation, counterfactual reasoning, and abstraction without collapse.

## 13 Geometry, Layout, and Gauge Freedom

Spherepop OS enforces a strict conceptual separation between semantic structure and geometric presentation. This separation is not a matter of implementation convenience but a foundational design principle intended to prevent accidental entanglement between meaning and visualization.

Geometric information—such as spatial layout, clustering, or visual proximity—is treated as non-causal metadata. Observers may attach, modify, or discard such information freely without affecting semantic state.

**Definition 13.1** (Layout Hint). *A layout hint is non-authoritative metadata associated with a semantic object that proposes a geometric or visual interpretation of that object.*

Layout hints are never consulted by the kernel during event interpretation. They do not influence equivalence, relations, or object existence. As such, they commute with all kernel operations.

**Axiom 13.1** (Geometric Gauge Freedom). *For any two assignments of layout hints, replaying the same event log yields identical semantic state.*

This axiom parallels gauge freedom in physics: multiple geometric representations may correspond to the same underlying structure. The semantic content of the system is invariant under changes of layout gauge.

This distinction is essential for collaborative environments. Different observers may adopt radically different visualizations suited to their tasks, cognitive preferences, or display constraints, while remaining in perfect semantic agreement.

## 14 Observers and Causal Synchronization

Observers in Spherepop OS are epistemic agents rather than causal actors. They receive information about state transitions but do not participate in authoritative execution.

**Definition 14.1** (Observer). *An observer is an external process that receives diffs derived from kernel state transitions and maintains a local semantic view.*

Observers are passive with respect to causation. They cannot modify kernel state directly and cannot introduce events into the authoritative log.

All diffs are delivered to observers in strictly increasing event order. This delivery discipline ensures that every observer witnesses a causally consistent history.

**Proposition 14.1** (Observer Convergence). *Any two observers that process the same prefix of diffs converge to semantically equivalent states.*

*Proof.* Since diffs are deterministically derived from kernel transitions and are applied in the same order, the resulting observer states differ only in non-authoritative metadata.  $\square$

Late-joining observers may be bootstrapped using snapshots followed by diffs, achieving eventual convergence without special handling.

## 15 Arbiter and Collaborative Commit Model

While observation is decentralized, authority in Spherepop OS is centralized by design.

**Definition 15.1** (Arbiter). *The arbiter is the sole entity authorized to append events to the authoritative log.*

Clients submit proposals to the arbiter, which validates them against the current kernel state and, if accepted, assigns them sequence identifiers and commits them to the log.

**Axiom 15.1** (Single Sequencer). *There exists exactly one arbiter per authoritative log.*

This axiom enforces total causal order. Unlike distributed consensus systems that permit partial ordering or eventual convergence, Spherepop OS commits to a single, linear history.

**Proposition 15.1** (Proposal Determinism). *Given identical kernel state and proposal content, the arbiter produces the same canonical event or rejection.*

This property ensures fairness, reproducibility, and auditability. The arbiter is not a policy engine but a deterministic mediator of semantic commitment.

## 16 Computational Complexity and Scalability

The computational properties of Spherepop OS follow directly from its kernel design. Because all operations are defined in terms of union-find and relation table updates, complexity bounds are well understood.

**Proposition 16.1** (Kernel Operation Complexity). *Object introduction is constant time. Equivalence induction via MERGE operates in inverse-Ackermann time amortized. Relation creation and removal scale logarithmically with the number of relations. Bulk collapse scales linearly with the size of the collapsed region, up to inverse-Ackermann factors.*

**Proposition 16.2** (Replay Complexity). *Replaying an event log of length  $\ell$  requires time linear in  $\ell$  up to inverse-Ackermann factors determined by equivalence maintenance.*

These bounds ensure that long-lived semantic histories remain tractable, even as abstraction accumulates. Importantly, abstraction tends to reduce effective state complexity by consolidating representatives, mitigating the cost of replay over time.

## 17 Rationale and Comparative Perspective

Spherepop OS occupies a position that is difficult to classify within existing computational paradigms. It is not a conventional operating system in the POSIX sense, as it does not manage hardware resources or schedule processes. It is not a database, as it does not support arbitrary mutation or query-driven updates. It is not a version control system, as speculation is continuous rather than episodic.

Instead, Spherepop OS is best understood as an operating system for semantic time. It manages the construction, identification, and stabilization of meaning over time through explicit events, equivalence, and replay.

In contrast to language-centric artificial intelligence systems, Spherepop OS does not begin with symbols. It begins with construction. Meaning emerges only after structure has been assembled, reused, and abstracted. This architectural choice aligns Spherepop OS with theories of intelligence that locate understanding in interaction and structure rather than in linguistic compression.

### Accumulation Regimes and the Stratification of Intelligence

A central contribution of Spherepop OS is that it renders visible, within a single formal substrate, multiple distinct regimes of knowledge accumulation that are often conflated in both artificial intelligence and cognitive science. These regimes correspond closely to the stratification identified by Yi Ma—phylogenetic, ontogenetic, social, and abstract—but are expressed here in strictly operational and architectural terms.

The key observation is that accumulation is not a unitary process. Different forms of intelligence arise from different mechanisms of compression, stabilization, and reuse, each operating on different timescales and with different degrees of internal autonomy. Spherepop OS provides a framework in which these differences are not merely conceptual but structurally enforced and empirically observable.

### 17.1 Phylogenetic Accumulation as Log Externalization

Phylogenetic accumulation corresponds to the slow incorporation of environmental regularities into the structure of an organism through evolutionary processes. In informational terms, this is a brutal and highly lossy form of compression, mediated by selection rather than inference.

Within the Spherepop OS framework, phylogenetic accumulation is best understood as *log externalization*. Structural commitments are fixed outside the active semantic substrate and appear to the system as immutable primitives. The system does not record the evolutionary history that produced these structures; it merely inherits them as given.

This regime is characterized by the absence of introspectable construction history. The reasons for a structure’s existence are not available within the system itself. As a result, phylogenetic knowledge is maximally stable but minimally revisable. It supports survival but not explanation.

Spherepop OS deliberately does not model this regime internally. Instead, it assumes a fixed kernel and primitive operation set, mirroring the way biological organisms inherit neural architectures without access to their evolutionary derivation.

## 17.2 Ontogenetic Accumulation as Event-Sourced Construction

Ontogenetic accumulation corresponds to individual learning through interaction with the environment. This is the regime in which Yi Ma locates animal and human intelligence at its core.

Spherepop OS is explicitly designed to operate at this level. Ontogenetic learning is modeled as the accumulation of events in an authoritative log, with semantic objects, relations, and equivalence classes emerging through replay. Crucially, this accumulation is internally driven. While external inputs may trigger events, the structural consequences of those events are determined by the system’s existing semantic organization.

Compression at this level is not mere reduction of data volume. It is the progressive elimination of redundant constructions through equivalence, reuse, and abstraction. The system learns not by memorizing observations but by stabilizing invariants across events.

Because the full construction history remains available, ontogenetic knowledge in Spherepop OS is introspectable, revisable, and replayable. This property sharply distinguishes it from both evolutionary inheritance and static database storage.

## 17.3 Social Accumulation as Shared Log Alignment

Social accumulation arises when multiple systems align their semantic substrates through shared symbols, conventions, and artifacts. Language, writing, and institutions belong to this regime.

From the perspective of Spherepop OS, social knowledge corresponds to partial log alignment across agents. Shared abstractions emerge when distinct event histories induce compatible equivalence classes and relations. Importantly, this does not require identical histories, only compatible semantic commitments.

This framing reveals why social knowledge is powerful but fragile. Because it operates above individual ontogenetic logs, it depends on continuous maintenance, interpretation, and repair. Misalignment does not appear as a logical contradiction but as semantic drift.

Spherepop OS supports this regime by design through its observer model. Multiple observers may maintain divergent views, layouts, and speculative branches while remaining anchored to a common authoritative core.

## 17.4 Abstraction and the Transition to Formal Knowledge

The highest regime identified by Yi Ma is the transition from empirical compression to abstract, deductive knowledge—science and mathematics. This transition represents a qualitative shift rather than a quantitative extension.

In Spherepop OS terms, this corresponds to the emergence of abstraction layers that operate primarily on representatives rather than on raw events. Once equivalence classes stabilize, reasoning can proceed by manipulating canonical objects without revisiting their construction histories.

This transition is visible in the log as a change in event character. Early logs are dominated by object creation and relation formation. Mature logs exhibit a preponderance of merges, collapses, and high-level relational rewriting. The system increasingly reasons about its own abstractions.

Crucially, this phase transition does not eliminate the underlying event substrate. Formal reasoning remains grounded in replayable construction, preventing abstraction from drifting into incoherence.

## 17.5 Why This Architecture Resolves the Compression–Abstraction Problem

Yi Ma poses a fundamental open question: what distinguishes compression from abstraction? Spherpap OS resolves this question structurally rather than definitionally.

Compression corresponds to a reduction in the number of events required to construct a semantic effect. Abstraction corresponds to a reduction in the number of distinct representatives required to account for those effects. Both processes are visible, measurable, and enforceable within the log.

This distinction explains why large language models, which operate primarily on already-compressed linguistic artifacts, struggle to achieve genuine understanding. They compress signals that have already undergone social and ontogenetic abstraction, without access to the underlying construction history.

Spherpap OS, by contrast, operates below language. It records the formation of meaning itself, rather than its symbolic residue. In doing so, it occupies the pre-linguistic substrate of intelligence that Yi Ma associates with animal cognition and world modeling.

## 17.6 Implications for Artificial Intelligence

By grounding intelligence in event-sourced semantic construction rather than symbolic manipulation, Spherpap OS avoids the trap of conflating memorization with understanding. Intelligence emerges not from scale alone, but from the capacity to accumulate, stabilize, and reorganize structure over time.

This perspective suggests that progress toward artificial general intelligence will require systems that expose and preserve their construction histories, rather than systems that merely optimize predictive performance on compressed corpora.

Spherpap OS offers a concrete architectural instantiation of this idea, providing a substrate on which learning, abstraction, and reasoning can be studied as unified phenomena.

### ectionA Field-Theoretic Interpretation of Semantic Construction

The architecture of Spherpap OS admits a natural interpretation in field-theoretic terms. This interpretation is not metaphorical but structural: the semantic state induced by the event log behaves as a dynamical field whose configuration evolves through localized interactions, stabilizes into attractors, and exhibits gauge freedom in its observational representations.

This perspective provides a unifying language for understanding learning, abstraction, and coherence within the system, and connects Spherpap OS to a broader tradition of field-based theories in physics, cognition, and cybernetics.

## 17.7 Semantic State as a Field Configuration

At any sequence index  $k$ , the kernel state  $\sigma_k = (O_k, U_k, R_k, M_k)$  induces a semantic configuration over the space of object handles. Rather than viewing this state as a static data structure, it is

more appropriate to regard it as a field assigning semantic potentials to relations between objects.

Relations act as couplings, equivalence classes as condensations, and metadata as gauge-dependent annotations. The replay operation corresponds to evolving this field forward in discrete time according to deterministic local update rules.

Importantly, the field is not defined over physical space but over a semantic manifold whose coordinates are given by object identity and relational context. The geometry of this manifold is emergent rather than prescribed.

## 17.8 Events as Local Field Perturbations

Each event in the log introduces a localized perturbation to the semantic field. Object creation introduces new degrees of freedom; relation creation alters coupling terms; merge and collapse events reduce dimensionality by identifying degrees of freedom.

These perturbations propagate through the field according to the kernel’s deterministic transition rules. Because all events are replayed in a total order, the resulting field evolution is well-defined and path-independent.

Learning, in this view, corresponds to the progressive reshaping of the field so that subsequent perturbations produce smaller global effects. The system becomes increasingly insensitive to noise and increasingly responsive to structurally meaningful changes.

## 17.9 Attractors and Semantic Stability

As event accumulation proceeds, the semantic field develops attractors: regions of configuration space toward which diverse event sequences converge. These attractors correspond to stable abstractions—equivalence classes and relational motifs that persist across contexts.

The existence of attractors explains how abstraction emerges without explicit symbolic reasoning. Abstractions are not imposed; they are the fixed points of semantic dynamics under repeated construction.

This view aligns with empirical observations in biological cognition, where concepts appear as stable patterns of activation rather than as discrete symbols. Spherpap OS provides an explicit operational substrate in which such patterns can be defined, observed, and replayed.

## 17.10 Gauge Freedom and Observational Equivalence

A central principle of field theories is gauge freedom: multiple representations may correspond to the same physical state. Spherpap OS exhibits an analogous property with respect to layout, geometry, and other non-causal metadata.

Different observers may assign different geometric embeddings or visual layouts to the same semantic field configuration without altering its authoritative content. These choices correspond to gauge transformations.

This interpretation clarifies why visualization, speculation, and exploration can be safely decoupled from semantic authority. The kernel operates on gauge-invariant quantities; observers are free to choose gauges appropriate to their tasks.

## 17.11 Relation to Cybernetics and Predictive Models

The field-theoretic view also connects Spherepop OS to cybernetic theories of intelligence. Feedback, error correction, and prediction correspond to the system’s capacity to compare expected semantic effects with observed perturbations and to reorganize its field accordingly.

However, unlike classical cybernetic models, Spherepop OS makes the construction history explicit. Prediction is not merely statistical but structural, grounded in replayable semantic commitments.

This distinction is crucial. It explains how intelligence can arise from compression without collapsing into mere memorization, and why abstraction requires access to generative history rather than surface regularities.

## 18 Conclusion

Spherepop OS presents a deterministic semantic operating system grounded in causal clarity, replayability, and structural abstraction. By elevating the event log to the sole source of authority and rigorously separating causes from observations, the system supports collaboration, introspection, and counterfactual reasoning without sacrificing consistency.

More importantly, Spherepop OS provides a concrete architectural answer to foundational questions in the theory of intelligence. Compression is revealed as reduction in causal construction; abstraction as reduction in semantic representatives. Both are enforced structurally and rendered visible in the system’s history.

By operating below language and symbolism, Spherepop OS occupies a pre-linguistic stratum of intelligence, corresponding to the level at which world models are formed through interaction and consolidation rather than description. In this sense, the system is not merely an operating system but a semantic substrate upon which more familiar cognitive and computational structures may be built.

## A Determinism and Confluence

**Theorem A.1.** *For any finite prefix of the authoritative event log, the resulting semantic state is unique.*

*Proof.* By induction on the length of the log prefix, using kernel determinism. □

## B Compression–Abstraction Duality

The dual roles of compression and abstraction are independent and jointly necessary. Compression governs the efficiency of construction; abstraction governs the stability of meaning. Neither alone suffices to produce robust intelligence.

## C Relation to Foundational Theories of Intelligence

Yi Ma’s principles of parsimony and self-consistency are instantiated architecturally within Spherepop OS. Norbert Wiener’s cybernetic emphasis on feedback and control is extended through explicit temporal replay and counterfactual branching.

## D Formal Distinction Between Compression and Abstraction

This appendix formalizes a distinction that is central to the theoretical claims of the paper: the difference between compression and abstraction. While these terms are often used interchangeably in the literature, they correspond to distinct structural transformations in Spherepop OS.

### D.1 Preliminaries

Let  $\ell = (e_1, \dots, e_n)$  be a prefix of the authoritative event log, and let  $\sigma(\ell)$  denote the kernel state obtained by replaying  $\ell$  from the empty initial state.

Let  $|\ell|$  denote the length of the log prefix, and let  $|O(\ell)|$  denote the number of object identifiers present in  $\sigma(\ell)$ . Let  $|U(\ell)|$  denote the number of equivalence classes induced by the union–find structure.

### D.2 Compression as Event Reduction

**Definition D.1** (Semantic Effect). *A semantic effect is an equivalence class of kernel states under observational equivalence, where two states are observationally equivalent if they induce the same canonical representatives and relations.*

**Definition D.2** (Compression). *Compression is the process of reducing the minimal log length required to produce a given semantic effect.*

Formally, compression occurs when there exists a log prefix  $\ell'$  such that  $\sigma(\ell')$  is observationally equivalent to  $\sigma(\ell)$  and  $|\ell'| < |\ell|$ .

Compression therefore operates on the temporal dimension of construction. It removes redundant or unnecessary events while preserving semantic outcomes.

### D.3 Abstraction as Representative Reduction

**Definition D.3** (Abstraction). *Abstraction is the process of reducing the number of distinct canonical representatives required to account for a semantic effect.*

Formally, abstraction occurs when, for a fixed semantic effect,  $|U(\ell')| < |U(\ell)|$ , typically as the result of merge or collapse events.

Abstraction operates on the structural dimension of the semantic field. It does not shorten the log but simplifies the ontology induced by the log.

## D.4 Independence of Compression and Abstraction

**Proposition D.1.** *Compression and abstraction are logically independent operations.*

*Proof.* Compression may occur without abstraction by removing redundant events that introduce objects which are later unused, without altering equivalence classes. Conversely, abstraction may occur without compression when merge events increase log length while reducing the number of representatives.

Therefore, neither operation implies the other.  $\square$

## D.5 Visibility in the Log

**Proposition D.2.** *The distinction between compression and abstraction is observable in the event log and kernel state.*

*Proof.* Compression is observable as a reduction in minimal log prefixes producing a given semantic effect. Abstraction is observable as a reduction in the cardinality of equivalence classes in  $\sigma(\ell)$ . Both quantities are explicitly computable from the log and kernel state.  $\square$

## D.6 Implications for Learning Systems

This formal distinction explains why systems that optimize for compression alone may fail to achieve abstraction. Statistical models that reduce data volume without altering underlying representational commitments compress signals without restructuring semantics.

By contrast, abstraction requires the system to identify and stabilize equivalences across contexts, an operation that cannot be performed solely at the level of surface compression.

Spherepop OS enforces this distinction architecturally, ensuring that abstraction remains grounded in replayable construction rather than in opaque parameter adjustment.

ectionCategory-Theoretic Structure of Spherepop Semantics

This appendix provides a categorical interpretation of the semantic machinery of Spherepop OS. The goal is not abstraction for its own sake, but to clarify which aspects of the system are structural invariants and which are implementation artifacts.

## D.7 Objects, Morphisms, and Construction Histories

Let  $\mathcal{C}$  be a category whose objects correspond to semantic object handles and whose morphisms correspond to construction relations induced by events. Each event introduces either a new object, a new morphism, or a constraint identifying existing objects.

The event log  $\ell = (e_1, \dots, e_n)$  induces a diagram  $D_\ell$  in  $\mathcal{C}$  whose shape is determined by the dependency structure of events. Replay corresponds to the progressive extension of this diagram.

Importantly, the category  $\mathcal{C}$  is not assumed to be small or finitely generated a priori; its finiteness at any given time is an emergent property of the log prefix.

## D.8 POP as Object Introduction

A POP event introduces a new object  $o$  into the category, corresponding to the free adjunction of an object. Categorically, this is a left adjoint to the forgetful functor that removes isolated objects.

This interpretation clarifies why object creation is irreversible in the authoritative log: left adjoints do not, in general, admit inverses.

## D.9 LINK as Morphism Introduction

A LINK event introduces a morphism between two objects, enriching the diagram without collapsing structure. Morphisms are directed and typed, corresponding to labeled arrows in  $\mathcal{C}$ .

LINK events preserve object identity and do not alter equivalence classes. They increase relational density without reducing dimensionality.

## D.10 MERGE and COLLAPSE as Coequalizers

MERGE and COLLAPSE events impose equivalence relations between objects. Categorically, these operations correspond to the formation of coequalizers.

Given objects  $o_1, o_2$  with parallel morphisms or semantic roles, a MERGE event enforces their identification by introducing a coequalizer object representing their quotient. COLLAPSE generalizes this to finite families.

The resulting quotient object is universal with respect to the imposed identification, explaining the confluence and order-independence of merge operations.

## D.11 Semantic State as a Colimit

For any log prefix  $\ell$ , the kernel state  $\sigma(\ell)$  corresponds to the colimit of the diagram  $D_\ell$ . Replay computes this colimit incrementally.

This perspective yields a concise explanation of determinism: colimits, if they exist, are unique up to isomorphism. Different replay strategies compute the same semantic object.

## D.12 Views as Functorial Projections

Derived views, snapshots, and diffs correspond to functors from  $\mathcal{C}$  into categories of representations, such as graphs, geometric spaces, or visual layouts.

These functors need not be faithful. Lossy projections are permitted, provided they preserve semantic equivalence. This formalizes the principle that views are observational and non-authoritative.

## D.13 Speculation as Diagram Extension

Speculative branches correspond to local extensions of the diagram  $D_\ell$  without committing to the colimit. Such extensions exist in a slice category over the base diagram and may be discarded without affecting the canonical colimit.

This explains why speculation is structurally safe: it never alters the universal property defining semantic state.

## E Thermodynamic Constraints on Semantic Construction

This appendix situates Spherepop OS within the framework of non-equilibrium thermodynamics. The purpose is to clarify the physical interpretation of irreversibility, determinism, and abstraction in the system.

### E.1 Event Logs as Irreversible Processes

The append-only nature of the authoritative log defines an arrow of time. Each committed event increases the length of the log and cannot be removed without violating determinism.

This irreversibility mirrors the second law of thermodynamics. The log plays the role of an entropy ledger, recording irreversible semantic work performed by the system.

### E.2 Replay as Reversible Computation

While log growth is irreversible, replay itself is reversible in the computational sense. Given a log prefix, the kernel can reconstruct past states exactly.

This separation mirrors the distinction between microscopic reversibility and macroscopic irreversibility in statistical mechanics. The kernel is a reversible interpreter acting on an irreversible history.

### E.3 Abstraction as Entropy Reduction

Merge and collapse events reduce the number of effective degrees of freedom in the semantic state. From an informational perspective, these operations correspond to entropy reduction within the semantic field.

However, this reduction is not free. It requires work in the form of explicit merge events, which increase log length. Thus, abstraction trades temporal entropy (longer history) for structural simplicity (fewer representatives).

This tradeoff mirrors Landauer’s principle: erasure of distinctions incurs a thermodynamic cost.

### E.4 Self-Consistency and Free Energy

A semantic state is self-consistent if replay of new events produces localized effects rather than global reorganization. Such states minimize semantic surprise and stabilize abstractions.

This condition parallels the minimization of free energy in predictive systems. The system maintains a structured internal model that absorbs perturbations efficiently.

Spherepop OS enforces this condition structurally rather than statistically. Self-consistency is not inferred but constructed through replayable commitments.

### E.5 Why Statistical Compression Is Insufficient

Purely statistical compression reduces description length but does not alter the generative structure responsible for producing observations. Such compression operates on already-formed signals.

By contrast, semantic abstraction in Spherepop OS operates at construction time, before signals are rendered. This explains why compression at the level of natural language or sensory data cannot substitute for pre-linguistic semantic construction.

## E.6 Non-Equilibrium Steady States

Over long histories, systems operating under Spherepop semantics tend toward non-equilibrium steady states characterized by stable abstractions and bounded perturbation propagation.

These states are neither static nor maximally entropic. They are sustained by continual semantic work, analogous to dissipative structures in thermodynamics.

## References

- [1] Y. Ma, *Learning Deep Representations of Data Distributions*, MIT Press, 2023.
- [2] N. Wiener, *Cybernetics: Or Control and Communication in the Animal and the Machine*, MIT Press, 1948.